

The Last Mile of Quantitative Research:

Automating Charts, Narratives and PDF Publication

Table of Contents

1.0 Where the story begins	3
2.0 What the script actually does	3
3.0 The Excel file drives everything	3
4.0 Three types of pages — and how each is assembled	4
4.1 Text-only pages	4
4.2 Table pages (CSV files)	4
4.3 Chart / image pages	4
5.0 Feature 1: Tables overlapping source text	4
6.0 Feature 2: Pages that need no chrome	5
7.0 Feature 3: Controlling page numbers from Excel	7
8.0 Feature 4: Story-Telling Through Custom Footers	8
9.0 The back page	8
10.0 What was deliberately left out	9
11.0 Outcome	9

1. Where the story begins

The original code—**TRTA_8_report_generator.py**—was a monolithic script that performed the entire pipeline in a single run. It fetched data from the database, calculated complex financial metrics (like XIRR, rolling returns, and maximum drawdowns), generated all the Matplotlib charts, and then immediately used ReportLab to place those charts onto a PDF page. While this sounded efficient, it meant that two vastly different domains—quantitative data analysis and document layout—were tightly tangled together.

In practice, this caused severe development friction. Because everything happened at once, a small layout fix could not be tested in isolation. For example, if you noticed that a data table was overlapping with the footer text on page 34 and you simply wanted to adjust the table's bottom margin, you had to re-run the entire 1,800-line **TRTA_8_report_generator.py** script. The script would unnecessarily re-fetch data, recalculate portfolio growth metrics, and regenerate dozens of PNG charts from scratch just to test a 2-millimeter layout adjustment. Conversely, a temporary database connection timeout or a minor calculation bug in an external module would completely halt the PDF assembly, meaning no report could be published at all.

The first decision was to separate them. Chart generation and financial computations would stay in their own dedicated files, producing static CSVs and PNGs. PDF assembly would become a dedicated, standalone script: **create_report_v2.py**. This paper is the story of what that assembly script had to solve.

2. What the script actually does

create_report_v2.py takes three inputs and produces one output.

Input / Output	What it is
JSON config file	Points to folders: images, charts, output. Provides logo name, cover image, report details Excel path.
Excel metadata file	One row = one page. Columns control title, subtitle, source text, chart filename, font sizes, page number toggle.
Charts / CSVs	Pre-generated PNG images and CSV files placed in a folder the config points to.

The script opens the Excel file, reads every row from top to bottom, and assembles the PDF one page at a time. It does not compute any financial numbers. It does not generate any charts. It only places things.

3. The Excel file drives everything

Every design element of the report—from the page layout (such as a standard chart with a title, a full-size bleed image, or a text-only page) to the title text, font sizes, page numbers, and footers—can be completely customized through the Excel file. Every decision the script makes is read directly from the corresponding Excel row for that page, ensuring there are no hidden defaults buried in the code that the user cannot override.

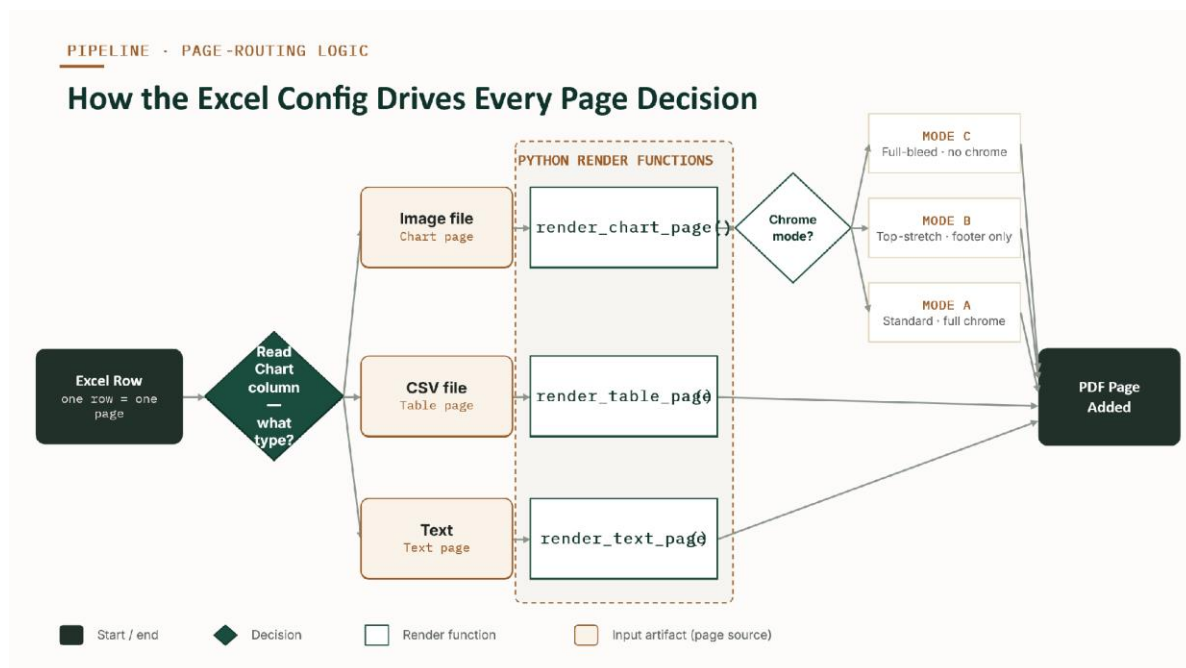


Figure 1 — For each Excel row, the script reads the 'Chart' column and routes to the correct renderer.

The 'Chart' column in the Excel file is the routing key. If it says 'text', the script renders a text-only page. If the value ends in '.csv', it renders a native table. Everything else is treated as an image filename.

4. Three types of pages — and how each is assembled

4.1 Text-only pages

When the 'Chart' column says 'text', the script draws the page header chrome (logo, title, subtitle, divider line, footer) and then places the descriptive text inside a paragraph frame. The text can be left-aligned, right-aligned, centred or justified — controlled by the 'Descriptive Text Alignment' column. Vertical position can also be set: top, centre or bottom of the content area.

4.2 Table pages (CSV files)

When the Chart column points to a CSV, the script parses the file and performs intelligent table truncation by stripping out redundant 'Label' columns to ensure an optimized presentation for the reader. It cleans the data by formatting dates (MMM-YY) and appending % signs to percentage values, then constructs a native ReportLab table with a structured grey header and grid lines.

Most importantly, the table is never flattened into an image — the script draws it directly as native text, ensuring all financial data in the final PDF remains fully selectable, searchable, and easy to copy.

4.3 Chart / image pages

When the Chart column contains an image filename, the script places the image onto the page. To make sure the image looks professional and leaves no empty white margins on the sides, the script stretches it to fill the available space exactly (`preserveAspectRatio=False`).

Depending on what we type into the Excel file, the script automatically chooses one of three ways to place the image:

- **Standard Mode:** If you provide a title, the script draws the full header (logo, title, subtitle, and divider line) at the top and the footer at the bottom. It then places your image perfectly in the space between them.
- **Top-Stretch Mode:** If you leave the title and other text fields empty but keep the page number turned on, the script removes the top header entirely. It stretches your image all the way from the very top edge of the page down to the footer band.
- **Full-Bleed Mode:** If you leave the text fields empty and also turn off the page number, the script removes both the header and the footer. It stretches your image to cover the entire page from edge to edge.

(Note: These three modes are visually demonstrated in Section 5).

5. Feature 1: Adaptive Page Layouts ("Clean" Pages)

Some pages in the booklet are designed to carry only a single large infographic or image—with no title, subtitle, source text, or descriptive text. If the script blindly drew the standard header chrome (the logo, circles, and divider lines) on every single page, it would waste valuable space and look disjointed, because the header graphics are meant to anchor a title.

To solve this, the script features an adaptive layout engine that detects "clean" pages automatically. If the script reads the Excel row and sees that all four text fields (Title Text, Subtitle Text, Source Text, Descriptive Text) are completely empty, it flags the page as clean. The script then adapts the output into one of three rendering modes:

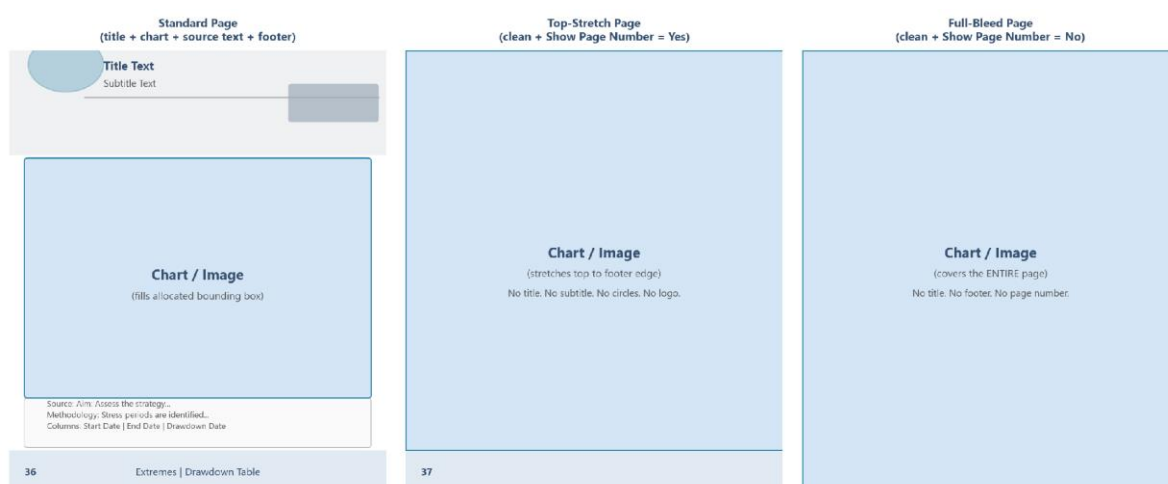


Figure 3 — The three layout modes: standard page, top-stretch (clean + page number), and full-bleed (clean + no page number).

- **Mode A — Standard page.** (Used when text fields are present). The script draws the title, subtitle, logo, circles, and footer. It then places the chart in the space remaining between the top divider line and the footer.

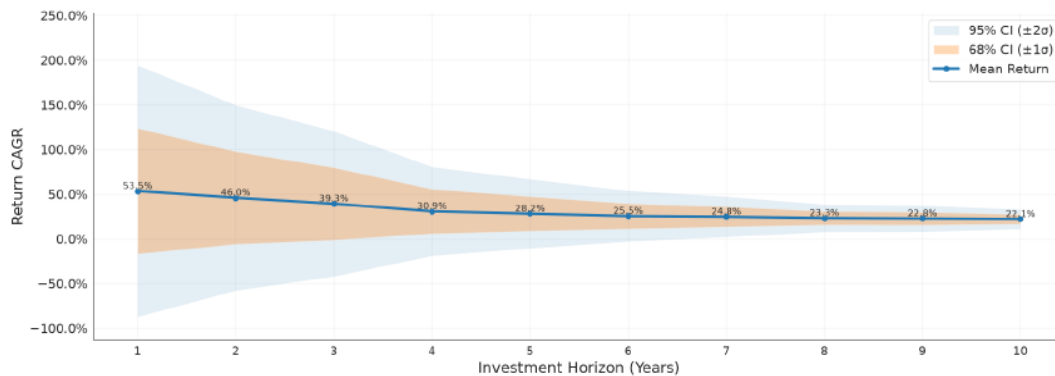


Strategy Rolling CAGR Confidence Intervals



Range of expected annualized returns at 68% and 95% confidence levels

Strategy Rolling CAGR Confidence Intervals (68% & 95%)



Aim: Assess the dispersion and predictability of the strategy's returns across different investment horizons.
Methodology: Rolling CAGRs are calculated for each investment horizon. The mean return is plotted alongside the 1-standard deviation (68% CI) and 2-standard deviations (95% CI) bounds to show the expected statistical range of outcomes.
Interpretation: The shaded regions indicate the probability of returns falling within a specific range. A narrower cone at longer horizons demonstrates that returns become much more predictable and less volatile as the holding period increases.
X-Axis: Investment Horizon (Years) | **Y-Axis:** Return CAGR | **Source:** ACE Equity, Fidelity Investments

46

Rolling Returns | Strategy Confidence Intervals

Mode A: Standard Page Aim

- **Mode B — Top-stretch page.** (Used when the Title and Subtitle text fields are absent). The script detects the clean page and removes the header graphics entirely. It keeps the footer band so the page number remains visible, and it stretches the chart upward so it starts from the absolute top edge of the page and ends perfectly at the footer band.

INVESTMENT PHILOSOPHY

Our world is full of patterns

Repetition, relationships and cycles reveal order across natural and human systems — and the same is true of markets.

Nature
Business
Markets

Order hides inside apparent randomness.

SEBI Registered Investment Adviser: INA000016056

Mode B: Top-Stretch Page

- **Mode C — Full-bleed page.** (Used when the Title and Subtitle text fields are absent, and 'Show Page Number' is set to No). The script removes both the header graphics and the footer band entirely. It stretches the chart to cover the entire page from edge to edge without any borders.

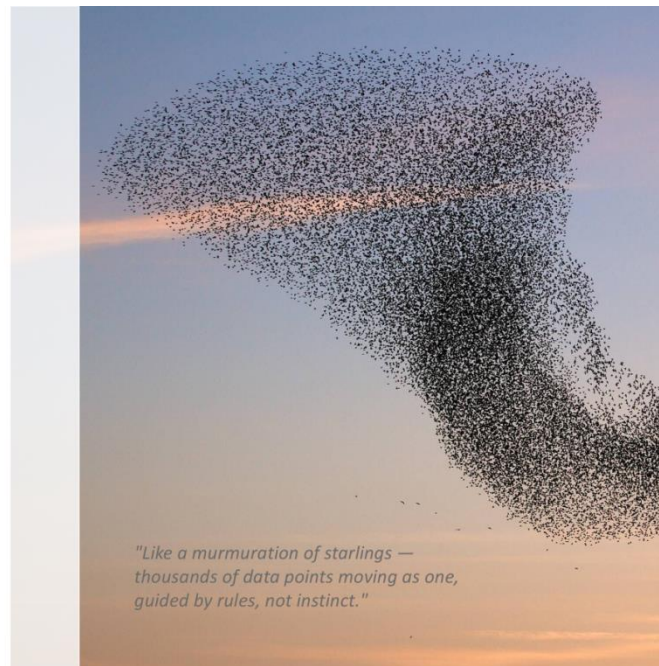


300,000+

25+

50M+

SEBI RIA



The script executes this in two steps:

- **Step 1 — Measure:** The script wraps the source text in a ReportLab Paragraph object and measures its height. This gives a precise number of points, not an estimate.
- **Step 2 — Calculate and Paginate:** Before drawing the table, the script calculates how much empty space remains between the title area at the top and the measured source text at the bottom. If only 14 rows fit safely in that space, it draws exactly 14 rows. The script then automatically creates a new page and moves the remaining rows there..

Because continuation pages do not carry the source text, the script switches to a standard bottom margin for those extra pages. It also automatically updates the page counter internally while drawing the extra tables, ensuring that the overall page numbers stay perfectly in sync.

7. Feature 3: Dynamic Page Number Control

The script draws page numbers automatically, starting the counter at 1 for the cover and incrementing it for every page, including any extra continuation pages generated by long tables.

However, displaying a page number on every single page creates a problem for cover slides, divider pages, or full-bleed infographics, which are designed to remain completely clean. Rather than hardcoding exception rules into the script for these specific pages, the system features a dynamic page number toggle controlled directly from the Excel file.

Show Page Number value	Page is clean?	What happens
Yes (or column absent)	No	Standard chrome + page number in footer.
Yes (or column absent)	Yes	Footer band drawn with page number only. (Mode B)
No (case-insensitive)	Yes	Footer band removed entirely. (Mode C)

The script simply reads a "Show Page Number" column in your Excel configuration. If you set it to "No", the script removes the page number for that specific page. If the column is absent from the Excel file entirely, the script falls back to a safe default and displays page numbers on every page. This means you can add or remove page number control just by updating the spreadsheet—no code changes are ever required.

8. Feature 4: Story-Telling Through Custom Footers

Often, a research report requires more than a repetitive, static footer on every page. Different sections might need a unique tagline, and specific charts might require contextual footnotes alongside the page number.

If the script hardcoded a single footer string for the entire document, creating a page-by-page narrative flow would be impossible. To solve this, the script introduces story-telling through customizable footers.

The system reads a dedicated "Footer Text" column from the Excel configuration file for every individual page. The script then takes that specific text and draws it directly into the footer band of that exact page. This allows you to weave a continuous narrative, add specific disclaimers only where the data requires them, or update the branding tagline midway

through the booklet—all controlled effortlessly from the spreadsheet without altering a single line of code.

9. The back page

The last page of every report is a contact and disclaimer page. The script draws it automatically after all content pages are assembled.

It reads contact details — email, phone, website, LinkedIn — from the first row of the Excel file. It places a logo image in the centre, then arranges the four contact items in a 2x2 grid below it, each with a small icon pulled from the Icon/ folder. Websites and LinkedIn URLs are embedded as live hyperlinks in the PDF. The disclaimer text, also from the Excel row, is rendered as a justified paragraph beneath a horizontal rule.

Below: attach a screenshot of the back page from an actual report.



Back Page: Contact & Disclaimer

10. What was deliberately left out

The separation of concerns matters as much as what was added. `create_report_v2.py` does none of the following, by design:

- **Chart generation.** No Matplotlib code, no axis configuration, no colour palettes. Charts arrive as finished PNG files.
- **Financial computation.** No XIRR, no drawdown calculation, no rolling return. Numbers come from the CSV files produced by other modules.
- **Data validation.** The script trusts its inputs. If a chart file is missing, it draws a red placeholder text and continues. It does not halt the run.
- **Excel writing.** The script reads from Excel. It never writes back to it.

This boundary is the reason the script stayed at 784 lines despite handling five distinct page types, three layout modes and dynamic pagination. Each part of the pipeline does one thing.

11. Outcome

Before `create_report_v2.py`, producing a 50-page research booklet required manual intervention after every run — adjusting table sizes, fixing overlaps, cropping or rescaling images. Each report took a material amount of time to review.

After the script reached its current form, the same 50-page report runs end-to-end without manual adjustment. The Excel file is the only thing that changes between strategies. Tables paginate correctly regardless of how many rows the CSV contains. Source text wraps correctly regardless of how long the methodology note is. Clean pages render full-bleed or top-stretch automatically based on two columns in the Excel file.

The story is that we treated the publication layer as a real engineering problem — defined its inputs, specified its behaviour precisely, and built it to be driven entirely by configuration rather than code changes.

FidelFolio Investments | Ravishekar Kumar | July 2026

This document describes the PDF assembly layer of the FidelFolio research publication pipeline.